



OPEN

# The effect of classical optimizers and Ansatz depth on QAOA performance in noisy devices

Aidan Pellow-Jarman<sup>2,9</sup>, Shane McFarthing<sup>2,3</sup>, Ilya Sinayskiy<sup>1,4✉</sup>, Daniel K. Park<sup>5,6</sup>, Anban Pillay<sup>7,9</sup> & Francesco Petruccione<sup>1,2,4,8</sup>

The Quantum Approximate Optimization Algorithm (QAOA) is a variational quantum algorithm for Near-term Intermediate-Scale Quantum computers (NISQ) providing approximate solutions for combinatorial optimization problems. The QAOA utilizes a quantum-classical loop, consisting of a quantum ansatz and a classical optimizer, to minimize some cost function, computed on the quantum device. This paper presents an investigation into the impact of realistic noise on the classical optimizer and the determination of optimal circuit depth for the Quantum Approximate Optimization Algorithm (QAOA) in the presence of noise. We find that, while there is no significant difference in the performance of classical optimizers in a state vector simulation, the Adam and AMSGrad optimizers perform best in the presence of shot noise. Under the conditions of real noise, the SPSSA optimizer, along with ADAM and AMSGrad, emerge as the top performers. The study also reveals that the quality of solutions to some 5 qubit minimum vertex cover problems increases for up to around six layers in the QAOA circuit, after which it begins to decline. This analysis shows that increasing the number of layers in the QAOA in an attempt to increase accuracy may not work well in a noisy device.

Many promising quantum algorithms, offering polynomial and exponential speed-ups over their classical counterparts, have been proposed<sup>1</sup>. These algorithms, including Grover's unstructured search algorithm<sup>2</sup> and Shor's algorithm for finding the prime factors of an integer<sup>3</sup>, have generated much excitement over the prospects of quantum computing. However, their practical realization is generally accepted to be a long term project due to constraints such as noise and state fidelity<sup>1</sup>. Error-correction schemes that would yield fault-tolerant quantum computers have been devised, but they require quantum computers with many more qubits than we have available at present<sup>4</sup>.

In the meanwhile, there is significant interest in quantum algorithms that are applicable to noisy intermediate-scale quantum (NISQ) computers currently available in the near to intermediate future<sup>4</sup>. These algorithms are predominantly variational and use hybrid quantum-classical routines to leverage existing quantum resources. They include the Variational Quantum Eigensolvers (VQE)<sup>5</sup> and the Quantum Approximate Optimization Algorithm (QAOA)<sup>6</sup>. As the number of the qubits in these near-term quantum computers increases, they become increasingly difficult to simulate with classical computers<sup>1</sup>.

The VQE and QAOA algorithms utilize parameterized quantum circuits  $U(\theta)$  to evolve the state of the Hamiltonian  $H$  representing the problem of interest. Using the expectation value  $\langle U(\theta) | H | U(\theta) \rangle$ , a classical optimizer is used to train the parameters of the quantum circuit. In this way, the algorithm uses the ansatz to prepare trial solutions to the problem, and the classical optimizer searches for better approximations of the ideal solutions<sup>5,6</sup>.

In QAOA, the ansatz is constructed from  $p$  layers of exponentiated cost and mixer Hamiltonians obtained from the problem cost definition<sup>6</sup>. As  $p \rightarrow \infty$ , the solution prepared by QAOA approaches the ideal solution, but, in the context of NISQ computing it is not feasible to utilize such deep circuits due to the effects of noise and state decoherence<sup>4,6</sup>. The noise in the NISQ computers also adversely affects the efficacy of the classical optimization procedure. The performance of the classical optimizer has recently been studied on the QAOA<sup>7</sup>,

<sup>1</sup> School of Chemistry and Physics, University of KwaZulu-Natal, Durban 4001, South Africa. <sup>2</sup>Qunova Computing Inc, Daejeon 34051, South Korea. <sup>3</sup>Department of Physics, Stellenbosch University, Stellenbosch 7600, South Africa. <sup>4</sup>National Institute for Theoretical and Computational Sciences (NITheCS), Stellenbosch, 7600, South Africa. <sup>5</sup>Department of Applied Statistics, Yonsei University, Seoul 03722, South Korea. <sup>6</sup>Department of Statistics and Data Science, Yonsei University, Seoul 03722, South Korea. <sup>7</sup>Centre for Artificial Intelligence Research (CAIR), Cape Town, South Africa. <sup>8</sup>Stellenbosch University, School of Data Science and Computational Thinking, Stellenbosch 7600, South Africa. <sup>9</sup>School of Mathematics, Statistics & Computer Science, University of KwaZulu-Natal, Durban 4001, South Africa. ✉email: sinayskiy@ukzn.ac.za

as well as in other variational quantum algorithms<sup>8</sup>. Optimization protocols of several variations of the QAOA have also been recently studied<sup>9</sup>.

In this work, we investigate classical optimizers and circuit depths  $p$  to find the optimal optimizer choice and ansatz depth for the minimum vertex cover problem under realistic device noise. We utilized a noise model sampled from the IBM Belem quantum computer to simulate the effects of noise on the efficacy of the algorithm. To the best of our knowledge, this is the first investigation of optimal circuit depth for QAOA with noise.

The remainder of the paper is structured as follows: Section "Quantum approximate optimization algorithm and minimum vertex cover problem" revises the details of the minimum vertex cover problem and the QAOA algorithm, Section "Classical optimizers and comparisons" describes the optimizers used, and the test methodology followed in this work, and Section "Results" presents our findings and discusses their significance.

## Quantum approximate optimization algorithm and minimum vertex cover problem

### Minimum vertex cover problem

The Minimum Vertex Cover problem is an example of a binary optimization problem that is NP-complete. A vertex cover of a graph  $G = (V, E)$ , is a set of vertices  $V' \subseteq V$ , such that for every edge  $e = (u, v) \in E$ ,  $u \in V' \cup v \in V'$ . The minimum vertex cover is a set of vertices  $V^*$ , that is the smallest possible set satisfying the above condition for a given graph  $G$ . The minimum vertex cover problem is to find a set  $V^*$ .

The minimum vertex cover problem can be formulated as the following binary optimization problem:

$$\text{Minimize: } \sum_{i \in V} x_i \quad (1)$$

$$\text{Subject to: } x_i + x_j \geq 1, \quad \forall (i, j) \in E \quad (2)$$

$$\text{and: } x_i \in \{0, 1\}, \quad \forall i \in V \quad (3)$$

In Fig. 1, we provide examples of graphs that illustrate the minimum vertex cover problem.

Binary optimization problems, like the minimum vertex cover problem, have their solutions encoded in a bit string and require an algorithm capable of finding the appropriate bit string to minimize the cost function. For the minimum vertex cover, each bit in the bit string corresponds to a vertex in the problem graph. A bit value of 1 indicates that the vertex is in the cover set, and a bit value of 0 indicates that the vertex is not in the cover set. The QAOA is one such quantum algorithm capable of finding an approximate solution in the form of a bit string, read out of the quantum device directly through measurement. Each qubit corresponds to a vertex in the graph, and the measured value of 0 or 1, forms a bit string solution for the problem.

### Quantum approximate optimization algorithm

The quantum approximate optimization algorithm (QAOA) is used to solve combinatorial optimization problems using a hybrid quantum-classical framework<sup>6</sup>. Many real-world problems can be formulated such that the solutions are  $N$ -bit binary strings of the form

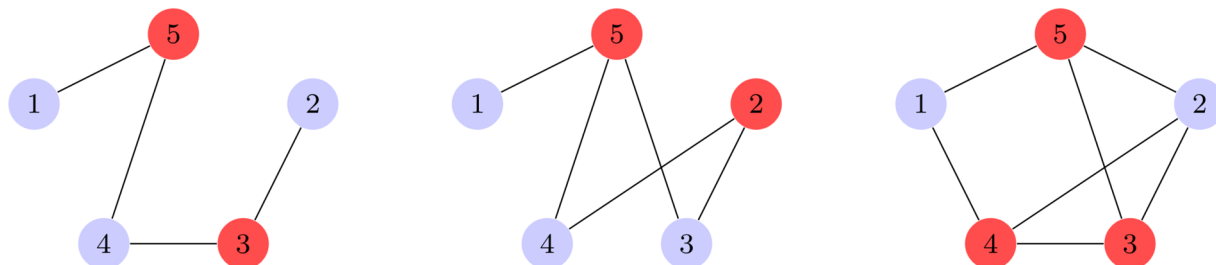
$$z = z_1 z_2 \dots z_N, \quad (4)$$

which minimize the classical cost function for  $m$  clauses,

$$C(z) = \sum_{\alpha=1}^m C_{\alpha}(z). \quad (5)$$

$C_{\alpha}(z) = 1$  if clause  $\alpha$  is satisfied by  $z$  and 0 otherwise<sup>6</sup>. Through the substitution of spin-operators  $\sigma_i^z$  for each  $z_i$  in  $z$ , one can build the cost Hamiltonian  $H_C$ ,

$$H_C = C(\sigma_1^z, \sigma_2^z, \dots, \sigma_N^z). \quad (6)$$



**Figure 1.** In the three graphs above, the red nodes show the set of vertices forming each graph's respective minimum vertex cover. Each edge in the graph under consideration must have, at least one vertex in the cover. The cover forms the minimum cover of a graph, when it contains the fewest number of vertices, whilst ensuring each edge is still incident to at least one.

The cost hamiltonian for the minimum vertex cost problem is given by:

$$H_C = A \sum_{(u,v) \in E} (1 - x_u)(1 - x_v) + B \sum_{v \in V} x_v \quad (7)$$

for an appropriate choice of  $A$  and  $B$ <sup>10</sup>. These constant terms are introduced because the minimum vertex cover problem contains hard constraints, which are not compatible with the QAOA, which solves only quadratic unconstrained binary optimization problems. The constant terms  $A$  and  $B$  refer to the weighting constants in the Hamiltonian.  $B$  weights the primary objective, minimizing the size of the vertex cover, while  $A$  weights a constraint or penalty term, that every edge have at least one of its vertices in the minimum cover. Since QUBO problems are unconstrained by nature, soft constraints in the form of penalty terms, forming soft constraints, are required.

Next, one can define a mixer Hamiltonian  $H_M$ ,

$$H_M = \sum_{j=1}^N \sigma_j^x. \quad (8)$$

Through the application of layers of alternating cost and mixer Hamiltonians to the initial state  $|+\rangle^{\otimes N}$ , an equally-weighted superposition of all states in the computational basis, the QAOA circuit from Fig. 2 is constructed<sup>6</sup>. This yields

$$|\psi_p(\vec{\gamma}, \vec{\beta})\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \dots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^{\otimes N}, \quad (9)$$

where  $p > 1$  is the number of layers in the circuit with  $2p$  parameters,  $\vec{\gamma}_i$  and  $\vec{\beta}_i$  with  $i = 1, 2, \dots, p$ . A classical optimizer can be used to alter the parameters, to minimize the expectation value,

$$F_p(\vec{\gamma}, \vec{\beta}) = \langle \psi_p(\vec{\gamma}, \vec{\beta}) | H_C | \psi_p(\vec{\gamma}, \vec{\beta}) \rangle. \quad (10)$$

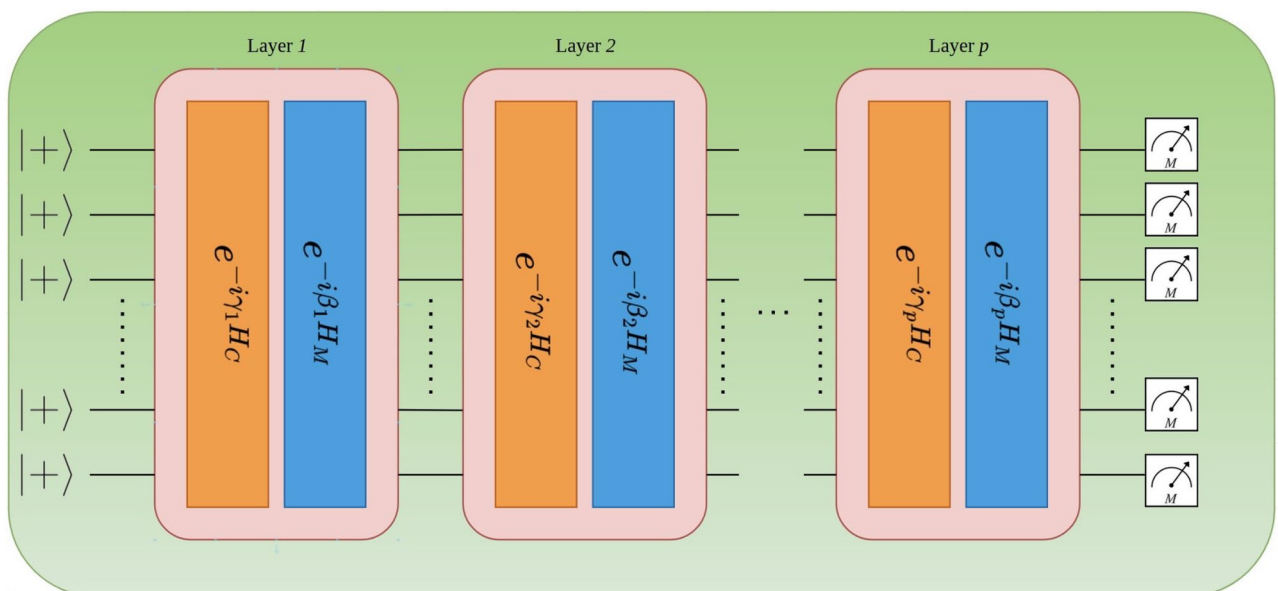
If  $\vec{\gamma}^*$  and  $\vec{\beta}^*$  minimize  $F_p$ , and if the value of the true solution is given by  $z^*$ , then the approximation ratio is given by,

$$\frac{F_p(\vec{\gamma}^*, \vec{\beta}^*)}{z^*}. \quad (11)$$

The approximation of the solution  $z^*$  can then be obtained through sampling of the state

$$|\psi_p(\vec{\gamma}^*, \vec{\beta}^*)\rangle, \quad (12)$$

prepared with the optimal parameters  $\vec{\gamma}^*$  and  $\vec{\beta}^*$ .



**Figure 2.** The QAOA circuit consists of  $p$  layers of the cost and mixer Hamiltonians,  $H_C$  and  $H_M$  respectively. The initial  $|+\rangle^{\otimes N}$  state is prepared and every qubit is measured after applying the QAOA circuit.

In a fully fault-tolerant setting, the performance of QAOA improves as  $p$  increases, however due to limitations in the hardware currently available, the behaviour of QAOA at lower values of  $p$  is of great interest<sup>11</sup>. Some previous works have investigated strategies for improving the performance of QAOA at these small  $p$ 's, such as using heuristic strategies for selecting the initial parameters, reducing the length of training required<sup>11</sup>.

Classical optimizers and comparisons

Classical optimizers

Variational quantum algorithms, QAOA included, find solution to given problems through the optimization of the ansatz parameters. There are a variety of classical optimizers that can be employed in these variational quantum algorithms. These optimizers can be grouped broadly into two categories: gradient-based and gradient-free. Gradient-based methods use gradient values during the optimization process. The gradient value evaluations can be done analytically for the expectation value of a quantum ansatz on a qubit hamiltonian, through the parameter-shift rule<sup>12</sup>, or more primitively, they can be estimated through finite differences. The parameter-shift rule is preferable because the analytic gradient is calculated through much larger variations of the ansatz parameters (and are therefore less susceptible to noise compared to finite differences), while still only requiring the original ansatz circuit in order to calculate the gradient (the same as finite differences). Gradient-free methods require only cost function evaluations, operating as a black-box optimizer. The optimizers compared are listed in Table 1.

This paper utilizes the parameter-shift rule for gradient calculations in all the gradient-based optimizers.

Classical optimizer comparison

The comparison of the aforementioned classical optimizers employed in the QAOA on the Min-Vertex Cover problem is described as follows.

The QAOA algorithm problem test set contains all 21 non-isomorphic 5-vertex graphs. The QAOA is applied to the Min-Vertex Cover problem ten times for each graph in the problem test set. This is repeated for each optimizer and each noise-level. Three levels of noise are considered, all resulting from the type of quantum simulation applied. These simulations are state vector, shot-based fault-tolerant, and shot-based with a sampled noise model, resulting in the noise-levels of noise-free, shot-noise and realistic quantum noise for currently existing quantum computers. The number of shots for the shot-based fault-tolerant and shot-based sampled-noise model is set to 10000. The number of iterations performed by the classical optimizers was limited by the number of function evaluations, which was set to 5000. When utilizing these algorithms on a quantum device, the number of function evaluations has a direct effect on the cost incurred as it determines the amount of device usage. The realistic noise model is sampled from the IBM Belem device. The implementation is done using PennyLane and the PennyLane-Qiskit package, in order to use Qiskit quantum backends and Noise-models.

QAOA Ansatz depth comparison

Once the most suitable classical optimizer is found, the next comparison finds the optimal depth of the QAOA ansatz circuit. In a state vector simulation of the QAOA algorithm, the accuracy increases as the number of layers increases, with the exact answer being achieved at the limit at which the number of layers approaches infinite. On a noisy quantum device, a deeper ansatz circuit will be more affected by noise, and hence the less the simulation will approximate the state vector simulation. This creates a trade-off between the theoretical accuracy increase achieved by increasing the number of layers, with the decrease in accuracy of the simulation caused by the decreased noise resistance that comes with the increased depth of the ansatz. More layers mean a more expressible ansatz (hence a better answer), but also more effects from noise (hence a worse answer).

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Gradient-Free                                                                   |    |
| Constrained Optimization by Linear Approximation <sup>α</sup> (COBYLA)          | 13 |
| Nelder-Mead <sup>α</sup>                                                        | 14 |
| Modified Powell's method <sup>α</sup>                                           | 15 |
| Simultaneous Perturbation Stochastic Approximation <sup>β</sup> (SPSA)          | 16 |
| Gradient-Based                                                                  |    |
| Broyden-Fletcher-Goldfarb-Shanno algorithm <sup>α</sup> (BFGS)                  | 17 |
| Limited-memory Broyden-Fletcher-Goldfarb-Shanno algorithm <sup>α</sup> (L-BFGS) | 18 |
| Conjugate Gradient method <sup>α</sup> (CG)                                     | 19 |
| Sequential Least Squares Programming <sup>α</sup> (SLSQP)                       | 20 |
| ADAM <sup>β</sup>                                                               | 21 |
| AMSGrad <sup>β</sup>                                                            | 22 |

**Table 1.** Table of Classical Optimizers Considered - The implementation of all classical optimizers are taken from the Scipy<sup>(α)</sup> and Qiskit<sup>(β)</sup> Python libraries, under the respective functions *scipy.optimize.minimize* and *qiskit.aqua.components.optimizers*. References are given in the far-right column.

For the three graphs in Fig. 3, we run a set of one hundred noisy simulations for each depth ranging from 1 to 10 layers, with the same noise model sampled in the optimizer comparison. We run the same simulations on a state vector simulator to show the theoretical convergence is achieved as the number of layer increases.

### QAOA Ansatz depth recommendation verification

Once the optimal depth is estimated from the experiments above, we seek to verify that these do indeed maximize the performance of the QAOA Algorithm on the Min-Vertex Cover problem for graphs of this size (5 qubits, 5 edges) in Fig. 4. We compare the QAOA with differing numbers of layers, and show that the solutions that are sampled from the QAOA with optimized parameters, are, on average, better when sampled from the QAOA with the optimal number of layers. It is expected that when the optimal number of parameters are used in the QAOA, the solutions sampled from the optimized QAOA ansatz will be on average better.

## Results

### Comparison of classical optimizers

The results for the different classical optimizers, gradient-free and gradient-based, for each ansatz depth from 1 layer to 5 layers are shown in Fig. 5 for the state vector, shot-based and noisy simulations in 5a–c respectively.

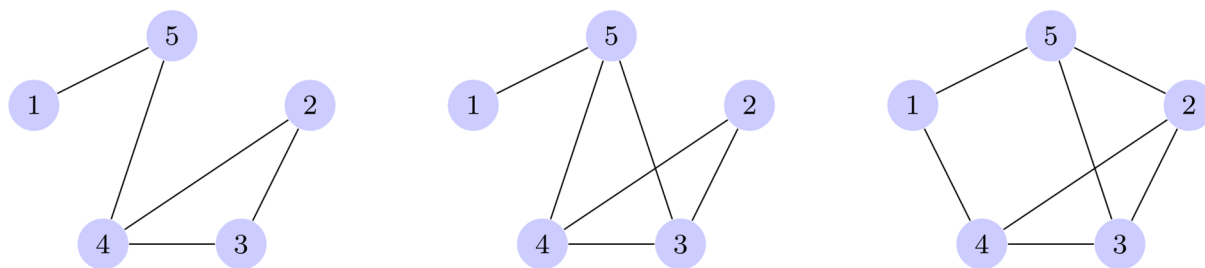
In the state vector simulation, all ten optimizers appear to perform similarly, with no major distinction between the gradient-free and gradient-based optimizers. There is a clear trend that more layers yields a better approximation ratio.

When shot-noise is accounted for, the differences between the optimizers become noticeable. SPSA slightly outperforms the other optimizers in this setting. The gradient-based optimizers all perform equally well, equivalent to the performance of Powell. COBYLA and Nelder-Mead are noticeably affected by the inclusion of shot-noise. For COBYLA, the approximation ratio for five layers is equivalent to the approximation ratio achieved with four layers, suggesting that this optimizer had trouble effectively making use of the extra parameters in the ansatz due to the shot-noise. All other optimizers show noticeable improvement with the increase in the number of layers.

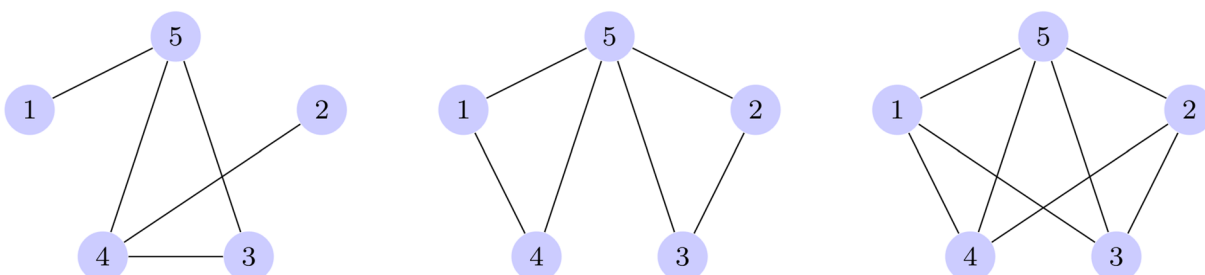
Finally, when the realistic noise model is incorporated into the simulation, the differences between classical optimizers become even more pronounced. SPSA is the best performing optimizer overall, achieving the best 4 and 5 layer approximation ratio in the presence of realistic noise-levels. COBYLA and Nelder-Mead are seriously negatively affected by noise, the approximation ratio for 5 layers is worse than that with 4 layers. All gradient-based methods show similar performance, equivalent to that of Powell.

Following these results, SPSA is used to run the noisy simulations for the QAOA ansatz depth comparison in Sect. "QAOA Ansatz depth comparison" and the ansatz depth verification in Sect. "QAOA Ansatz depth recommendation verification". COBYLA is used for the accompanying state vector simulations as it was found to be the best optimizer in the state vector simulation.

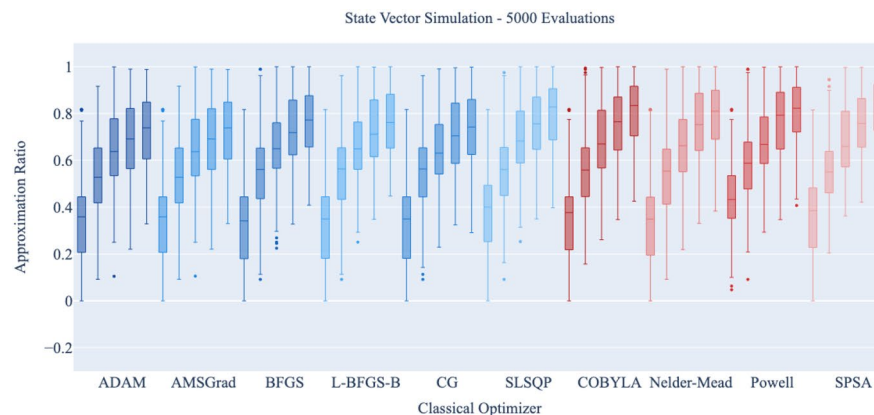
As an interesting note, when limiting the number of function evaluations permitted by the classical optimizer, the gradient-based optimizers' ability to accurately converge is severely affected. Thus, when utilizing algorithms with an ansatz based on Hamiltonian simulation on quantum hardware, it is not recommended that one make use of these gradient-based optimizers. As these optimizers require more calls to the quantum device to evaluate



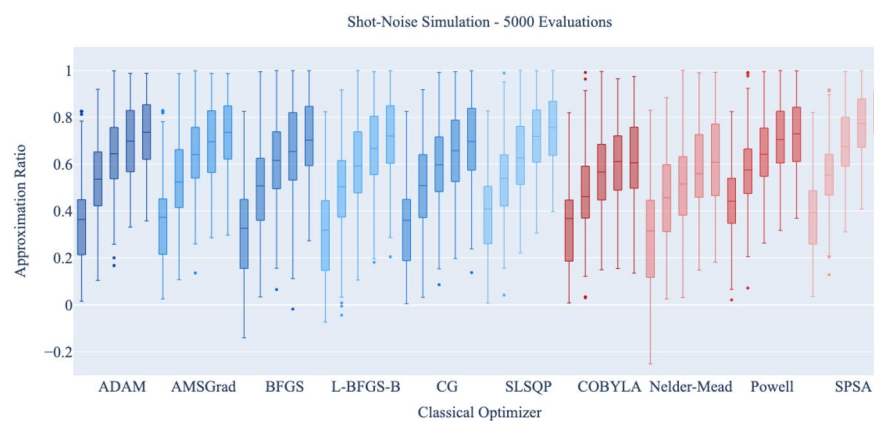
**Figure 3.** The three graphs used in the QAOA depth comparison, referred to as graph 1, 2 and 3 respectively.



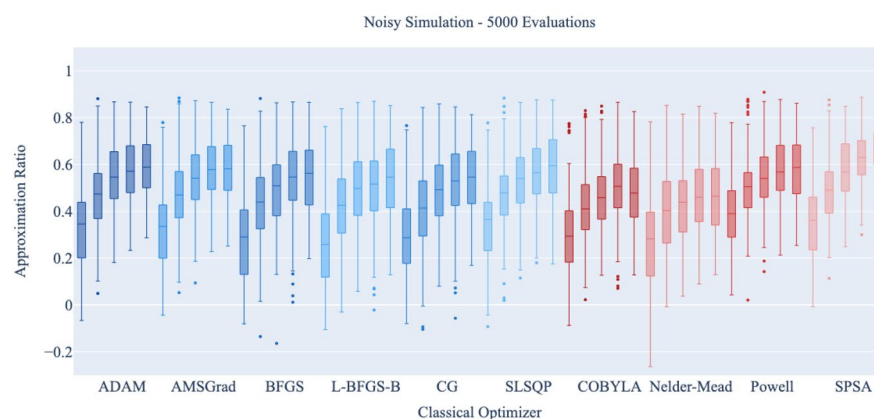
**Figure 4.** The three graphs used in the QAOA depth recommendation comparison, referred to as graph 4, 5 and 6 respectively.



(a)



(b)



(c)

**Figure 5.** A comparison of the approximation ratio using 10 classical optimizers on the QAOA, each having depths 1 to 5 from left to right, using a statevector, shot-based, and noisy simulation in (a–c) respectively. Each box is made up of the 10 runs for each of the 21 non-isomorphic graphs. The three graphs capture the change in performance as one moves from application in a noiseless regime, to the application of the methods in the presence of realistic noise from a quantum device. This demonstrates the change in performance that could be expected on a real device and it can be observed that as more noise is introduced to the system, there is a decrease in the performance of the classical optimizers, with some optimizers being more affected than others. Each problem instance optimization is done with a maximum of 5000 cost function evaluations.



the gradient, limiting this causes the observed degradation in performance. For the different types of variational circuits similar results were reported in Refs.<sup>23,24</sup>.

### QAOA Ansatz depth comparison

Figures 6, 7 and 8, give the comparison between the state vector and noisy simulations for the three graphs respectively.

In the state vector simulations, the expected trend is clearly apparent; that the approximation ratio of the QAOA algorithm improves steadily as additional layers are added. It is also clear that adding another layer seems to improve the approximation ratio less than the addition of the layer before. There is a diminishing return on accuracy increase with the number of layers added.

In the noisy simulations, as the number of layers increases, the effect of noise in the simulation becomes apparent. With the best approximation ratio achieved at around six layers for all graphs. Once this best approximation ratio is achieved, additional layers then begin to slowly worsen the approximation ratio.

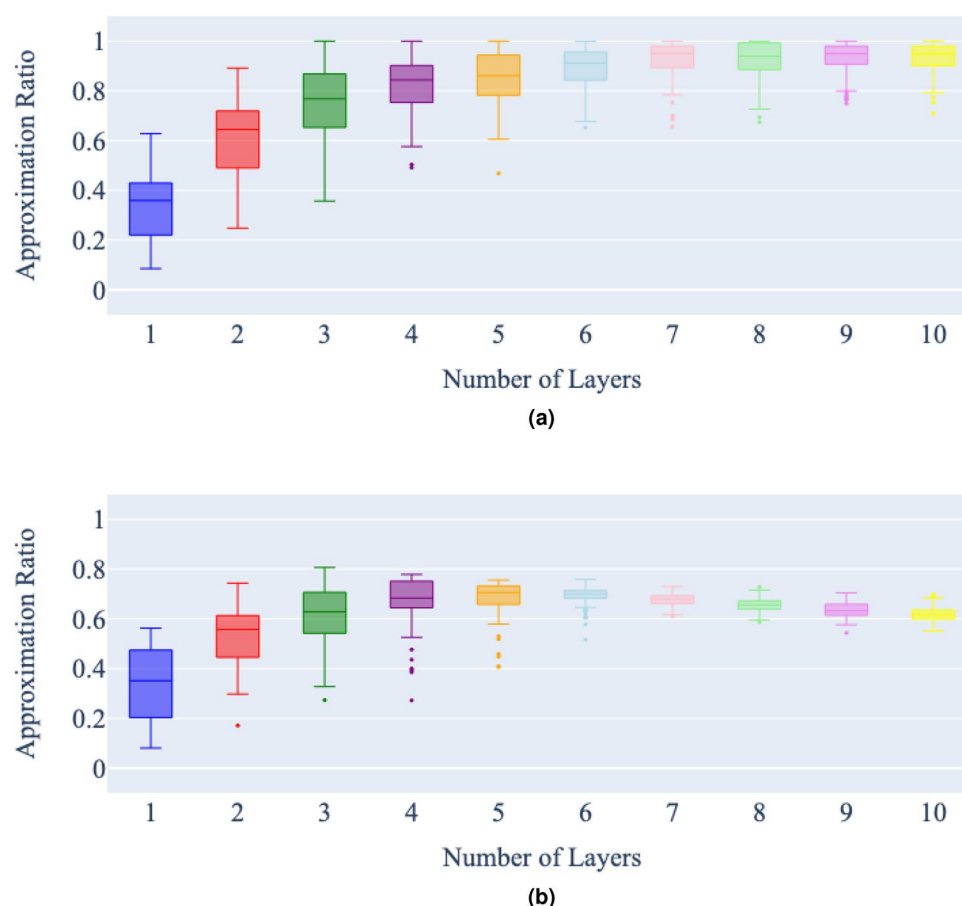
### QAOA Ansatz depth recommendation verification

Figures 9a–c show the probabilities of sampling the correct solution for the minimum vertex cover problem on graphs 4, 5 and 6, for the noise and state vector simulation respectively.

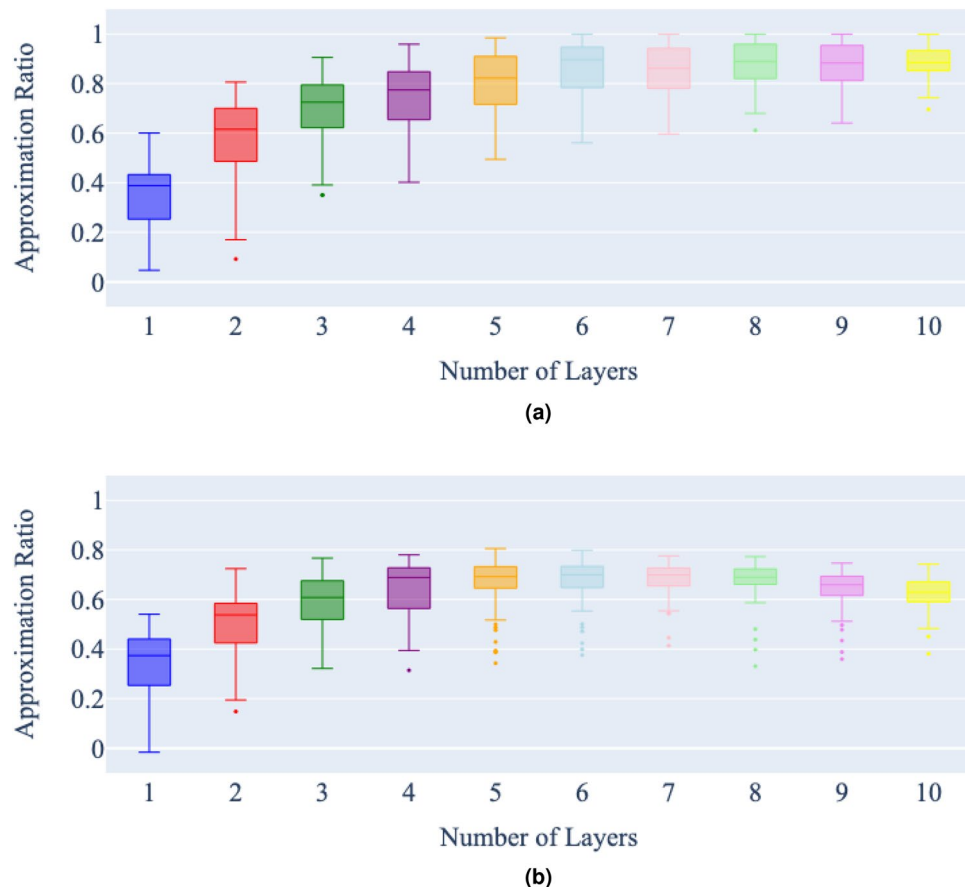
It is clear from these graphs, that 6 layers appears to be optimal for these instances of the QAOA too, allowing the correct solution for the minimum vertex cover on these graphs to be sampled with the greatest probability. After 6 layers, additional layers decrease the probability of sampling the correct solution for the minimum vertex cover problem.

### Discussion and conclusion

The results from Sect. "Comparison of classical optimizers", the comparison of classical optimizers in the QAOA, show that the choice of classical optimizer has a significant effect on the algorithm in the presence of noise. A classical optimizer's performance in a state vector simulation does not accurately reflect its performance in a realistic noise setting. It appears that SPSA is the best classical optimizer for current levels of noise. This is a result



**Figure 6.** Number of layers vs approximation ratio of a state vector (a) and noisy (b) simulation of QAOA for the minimum vertex cover on graph 1, with a maximum of 5000 cost function evaluations per problem instance. Each bar represents 100 runs of the QAOA for each number of layers, for the same minimum vertex cover problem.



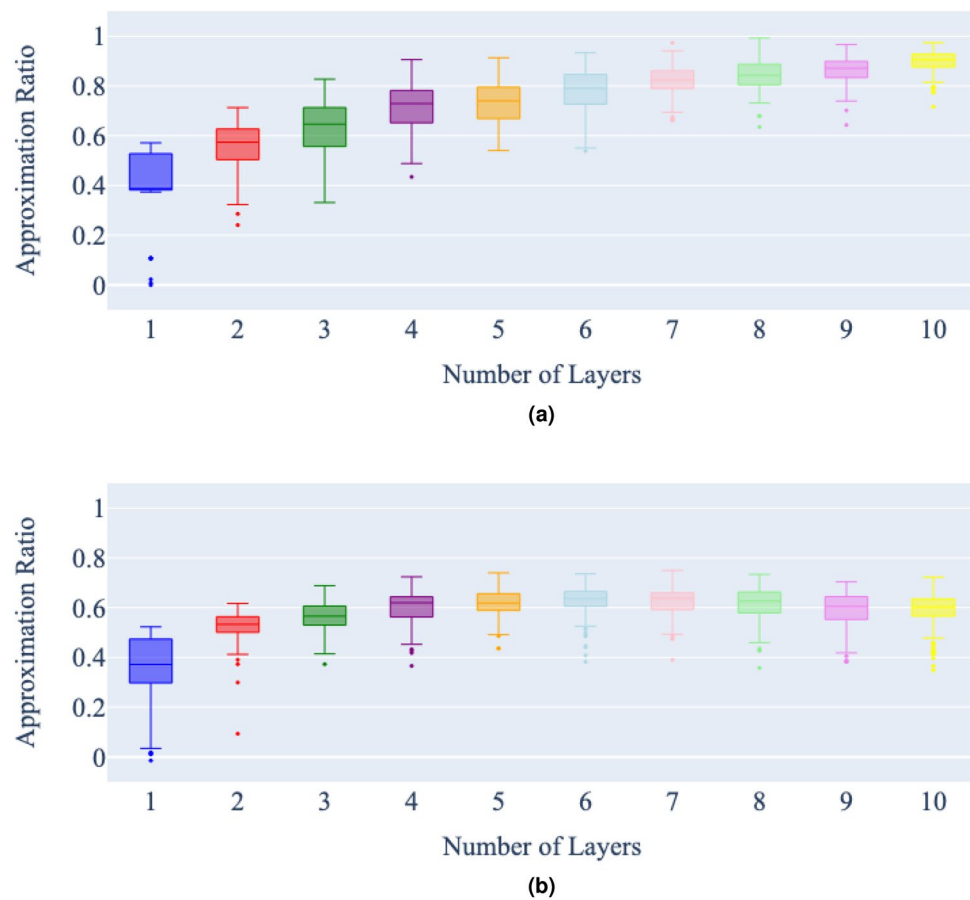
**Figure 7.** Number of layers vs approximation ratio of a state vector (a) and noisy (b) simulation of QAOA for the minimum vertex cover on graph 2, with a maximum of 5000 cost function evaluations per problem instance. Each bar represents 100 runs of the QAOA for each number of layers, for the same minimum vertex cover problem.

of both its built-in stochastic nature making it more resistant to noise, and its efficient gradient approximation requiring only two cost function evaluations for any ansatz. These results are similar to those found in Ref.<sup>8</sup>, where SPSSA was found to be the best classical optimizer in the noisy simulation, while COBYLA and Nelder-Mead were found to be the worst.

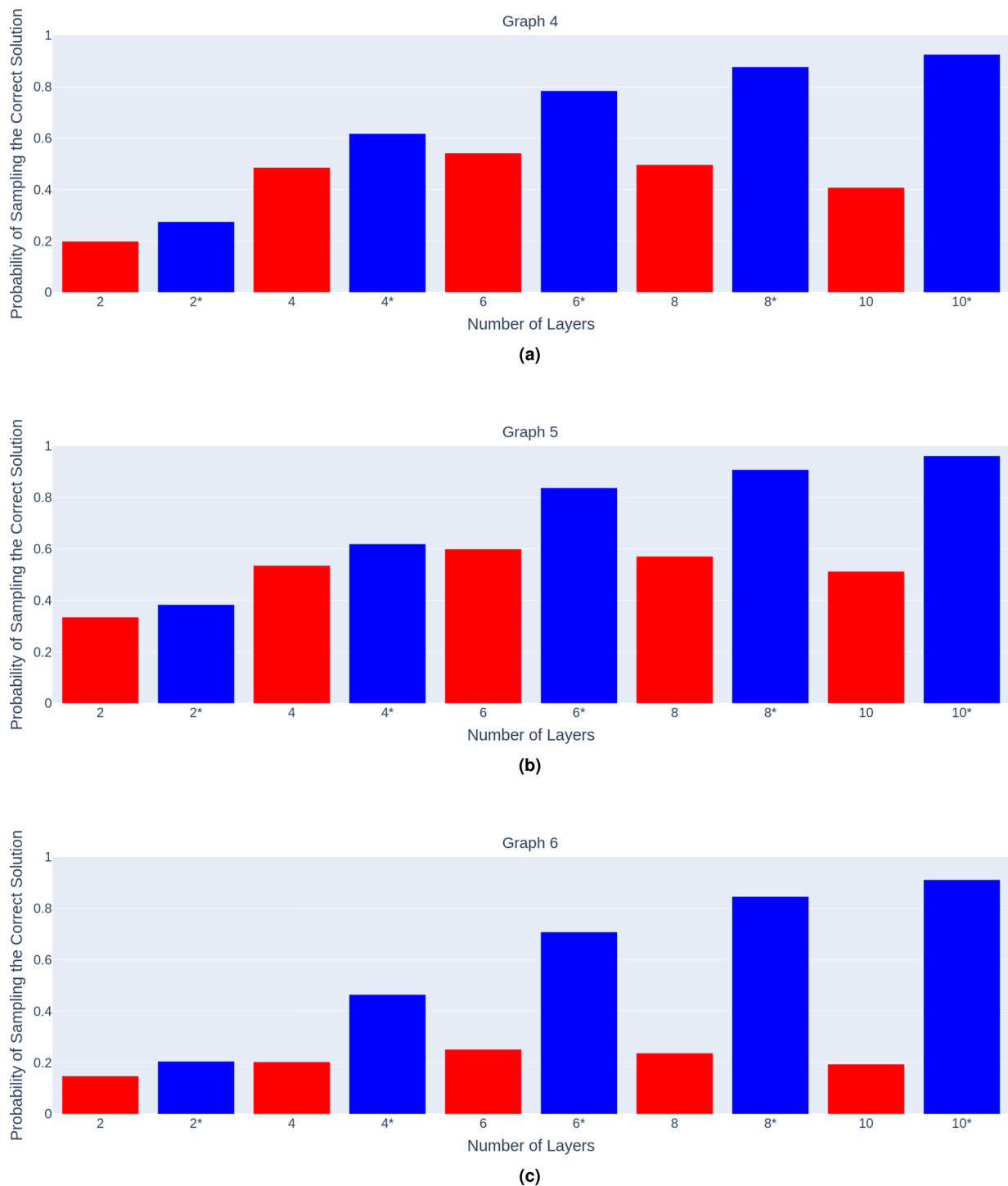
The results from Sect. "QAOA Ansatz depth comparison", the comparison of depths for the QAOA ansatz, show that while in theory the more layers present in the QAOA the greater the accuracy, it is actually the case that in the presence of noise in the circuit, there is actually an optimal number of layers that provide the greatest accuracy. It is therefore important to use the correct number of layers in order to utilize the QAOA algorithm to its full potential. Previous works have suggested other guidelines for ansatz depth based on factors such as time complexity of execution<sup>25</sup>. Section "QAOA Ansatz depth recommendation verification" shows that the probability of sampling the correct solution for the QAOA problem is also greatest at the optimal number of layers, and decrease as the number of layers increase beyond that.

It is left as a future work to fully characterize the trade-off level of noise in the circuit and the accuracy improvement yielded by adding additional layers to the QAOA. It is hoped that it will then be possible to estimate the optimal number of layers for a QAOA circuit for a given problem and a device's noise level.





**Figure 8.** Number of layers vs approximation ratio of a state vector (a) and noisy (b) simulation of QAOA for the minimum vertex cover on graph 3, with a maximum of 5000 cost function evaluations per problem instance. Each bar represents 100 runs of the QAOA for each number of layers, for the same minimum vertex cover problem.



**Figure 9.** Number of layers vs probability of sampling the correct solution to the minimum vertex cover problem on each graph respectively (a–c), for both noisy (red) and statevector (blue) simulations.

### Data availability

The data is available at <https://github.com/aidanpellow/qaoa>.

Received: 2 August 2023; Accepted: 2 July 2024

Published online: 11 July 2024

### References

1. Nielsen, M. A., and Chuang I., Quantum computation and quantum information (2002).
2. Grover, L. K., A fast quantum mechanical algorithm for database search. In: Proc. Twenty-eighth annual ACM symposium on Theory of computing, 212–219 (1996).

3. Shor, P. W. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science* (ed. Shor, P. W.) 124–134 (IEEE, 1994).
4. Preskill, J. Quantum computing in the NISQ era and beyond. *Quantum* **2**, 79 (2018).
5. Peruzzo, A. *et al.* A variational eigenvalue solver on a photonic quantum processor. *Nat. Commun.* **5**(1), 4213 (2014).
6. E. Farhi and J. Goldstone, A Quantum Approximate Optimization Algorithm. [arXiv:1411.4028](https://arxiv.org/abs/1411.4028) (2014).
7. Fernández-Pendàs, M. *et al.* A study of the performance of classical minimizers in the Quantum Approximate Optimization Algorithm. *J. Computat. Appl. Math.* **404**, 113388 (2022).
8. Pellow-Jarman, A. *et al.* A comparison of various classical optimizers for a variational quantum linear solver. *Quantum Inf. Process.* **20**(6), 202 (2021).
9. Wenyang Qian *et al.*, Comparative study of variations in quantum approximate optimization algorithms for the Traveling Salesman Problem. [arXiv:2307.07243](https://arxiv.org/abs/2307.07243).
10. Pelofske, E., Hahn, G. & Djidjev, H. Decomposition algorithms for solving NP-hard problems on a quantum annealer. *J. Signal Process. Syst.* **93**, 405–420 (2021).
11. Zhou, L., Wang, S. T., Choi, S., Pichler, H. & Lukin, M. D. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. *Phys. Rev. X* **10**(2), 021067 (2020).
12. Wierichs, D., Izaac, J., Wang, C. & Yen-Yu, Lin C. General parameter-shift rules for quantum gradients. *Quantum* **6**, 677 (2022).
13. M. J. D. Powell, A direct search optimization method that models the objective and constraint functions by linear interpolation. *Adv. Optim. Numer. Anal.* 51–67 (1994).
14. Nelder, J. A. & Mead, R. A simplex method for function minimization. *Comput. J.* **7**(4), 308–313 (1965).
15. Powell, M. J. D. An efficient method for finding the minimum of a function of several variables without calculating derivatives. *Comput. J.* **7**(2), 155–162 (1964).
16. Spall, J. C. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Trans. Autom. Control* **37**(3), 332–341 (1992).
17. Fletcher, R. A new approach to variable metric algorithms. *Comput. J.* **13**(3), 317–322 (1970).
18. Byrd, Richard H., Lu, Peihuang, Nocedal, Jorge & Zhu, Ciyou. A limited memory algorithm for bound constrained optimization. *SIAM J. Sci. Comput.* **16**, 1190–1208 (1995).
19. Hestenes, M. R. & Stiefel, E. Methods of conjugate gradients for solving linear systems. *J. Res. Natl. Bureau Standards* **49**, 409–35 (1952).
20. Boggs, Paul T. & Tolle, Jon W. Sequential quadratic programming. *Acta Numer.* **4**, 1–51 (1996).
21. D. P. Kingma and J. L. Ba, ADAM: A Method for Stochastic Optimization. [arXiv:1412.6980](https://arxiv.org/abs/1412.6980). (2017).
22. S. J. Reddi, S. Kale and S. Kumar, On the Convergence of ADAM and Beyond. [arXiv:1904.09237](https://arxiv.org/abs/1904.09237). (2018).
23. Grimsley, H. R. *et al.* An adaptive variational algorithm for exact molecular simulations on a quantum computer. *Nat. Commun.* **10**, 3007 (2019).
24. Solinas, P., Caletti, S. & Minuto, G. Quantum gradient evaluation through quantum non-demolition measurements. *Eur. Phys. J. D* **77**, 76 (2023).
25. Hadfield, S. *et al.* From the quantum approximate optimization algorithm to a quantum alternating operator Ansatz. *Algorithms* **12**(2), 34 (2019).

## Acknowledgements

Support from the NICIS (National Integrated Cyber Infrastructure System) e-research grant QICSI7 is kindly acknowledged. This research is also supported by the Yonsei University Research Fund of 2024 (2024-22-0147), the National Research Foundation of Korea (Grant No. 2022M3E4A1074591), and the KIST Institutional Program (2E32941-24-008).

## Author contributions

D.K.P., A.P. and I.S. conceived the experiments, A.P. conducted the experiment(s), A.P. and I.S. analysed the results. A.P., S.M. and I.S. wrote the paper. All authors reviewed the manuscript.

## Competing interests

Francesco Petruccione the Chair of the Scientific Board and Co-Founder of QUNOVA computing. Aidan Pellow-Jarman and Shane McFarthing are employees of Qunova Computing. The authors declare no other competing interests.

## Additional information

**Correspondence** and requests for materials should be addressed to I.S.

**Reprints and permissions information** is available at [www.nature.com/reprints](http://www.nature.com/reprints).

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2024